CISC

- Muitas instruções
- Versatilidade
- Complexidade (quando comparado com RISC)
  - Pode usar microprogramas internamente no processador
    - Para executar instruções mais complexas
    - Os programadores não têm acesso às microinstruções
- Múltiplos modos de endereçamento
- Nasceu em uma época que memórias eram caras e pequenas
  - Compiladores deveriam gerar código compacto e eficiente
    - Instruções do processador deveriam executar várias funções

x86 (CISC)

- O projeto inicial visava eficiência do software de alto nível
- Conjunto de instruções para facilitar a construção de compiladores
- Suporte de estruturas de alto nível
  - Ex.: recursos para stack frames, arrays, strings, funções aninhadas, etc
- Ao longo do tempo novas instruções foram sendo acrescentadas
  - Também: preocupação com compatibilidade com software antigo
- Na microarquitetura P6 (Pentium Pro) de 1995
  - As instruções x86 foram convertidas internamente em microoperações mais simples, no estilo RISC
  - Porém o programador não tem acesso

Desenvolvimento da arquitetura CISC

- Alguns aspectos que se destacam
  - Utilização de microcódigo
  - Instruções completas e eficientes
  - Instruções complexas auxiliando compiladores e ling. de alto nível
  - Foi sendo desenvolvido ao longo do tempo para PCs
    - Ex.: não tem tanta preocupação com consumo de energia
- Regras usualmente seguidas no desenvolv. da arquitetura CISC
  - Instruções com campos de origem e destino
    - Ex.: sem precisar de instruções adicionais para para ler/gravar entre memória e registradores
  - 3 modos de transferência de dados
    - Registrador para registrador
    - Registrador para memória
    - Memória para registrador

(continua)

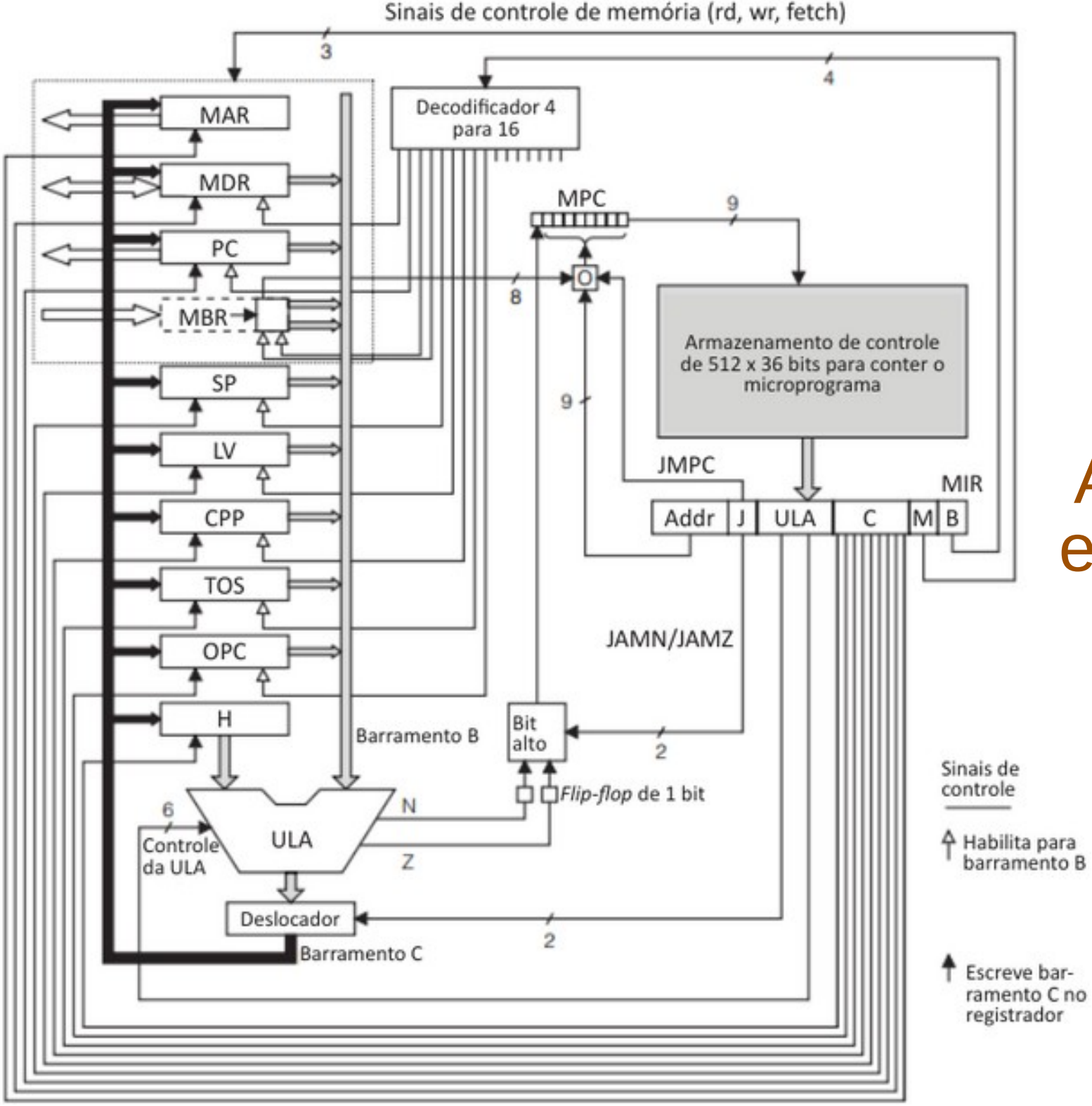
(continuação)

- Diferentes modos de endereçamento para a memória
- Instruções com largura variável
  - Quantidade de bytes variando de acordo com o modo de endereçamento
- Instruções executadas em múltiplos ciclos de clock
  - Varia conforme a largura da instrução
- Processador possui poucos registradores
  - Instruções acessam diretamente a memória sem passar por registradores
- Existência de registradores especializados
  - Controle, segmento, ponteiros de pilha, etc.

- RISC veio depois do CISC
  - Procurou resolver alguns problemas que aconteciam com CISC
    - Ex.: a grande quantidade de instruções afetava o desempenho global
    - A partir do P6, de qualquer forma, as instruções complexas são convertidas
    - Foi sendo desenvolvido ao longo do tempo também para aplicações móveis

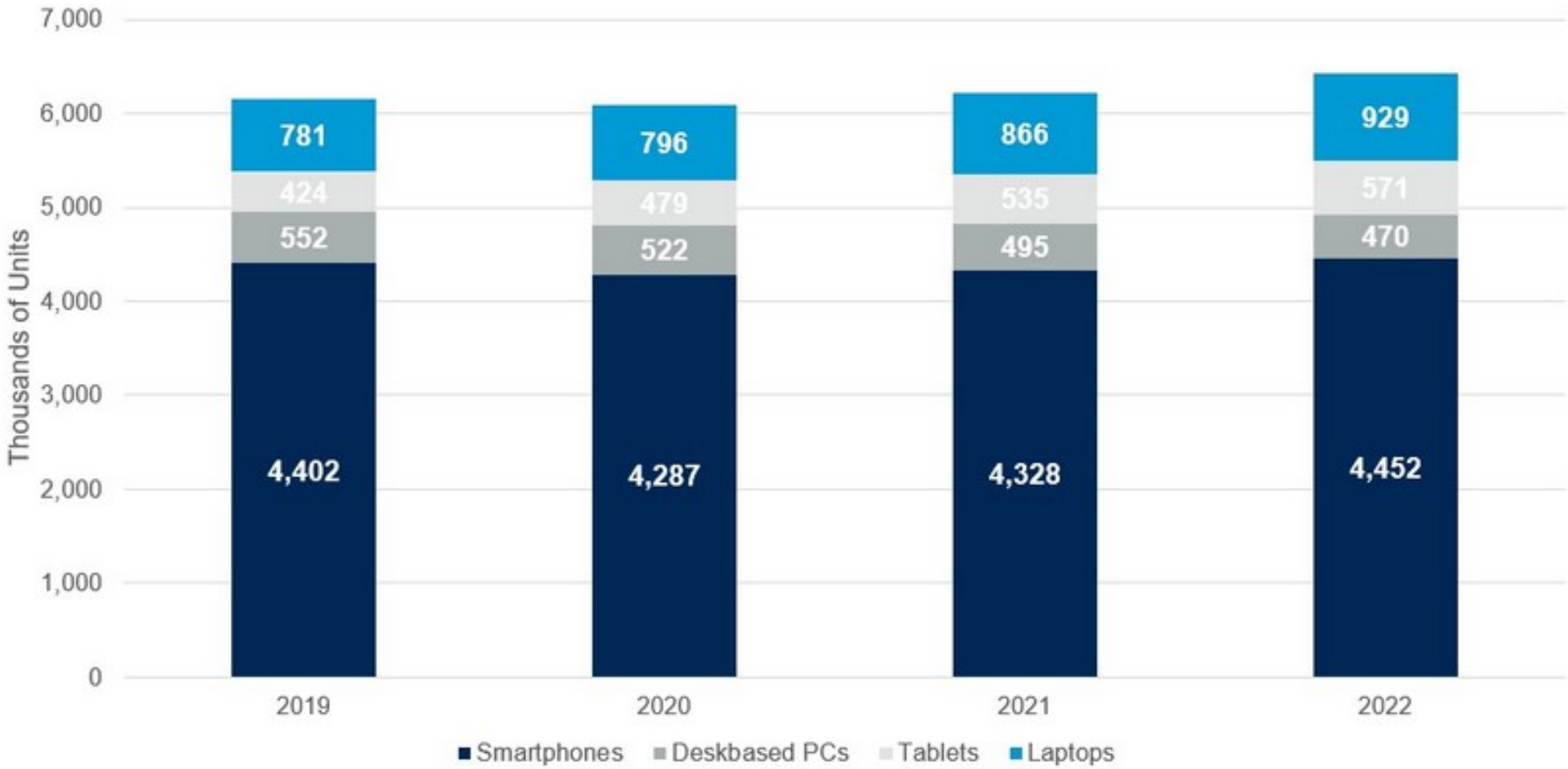
Pontos problemáticos da CISC

- Diferenças de velocidade entre memória e processador
- Microcódigo
  - A ideia da RISC é usada de qualquer forma
  - Por que não simplificar e usar o RISC direto?
- Grande quantidade de instruções
  - Apesar do suporte a compiladores e linguagens de alto nível, aumenta a complexidade e diminui o desempenho global
  - Apesar do código parecer mais denso e com poucas instruções, as microinstruções aumentam a quantidade total de passos de execução
- Necessidade de compatibilidade com processadores anteriores
  - RISC procura amenizar o problema usando simplicidade



Arquitetura educacional Mic-1

RISC vs CISC (2021)



https://tiinside.com.br/22/04/2021/base-instalada-de-dispositivos-chegara-a-62-bilhoes-este-ano-estima-gartner/

Características	RISC	CISC
Complexidade e quantidade de instruções	Instruções simples e em número reduzido.	Muitas instruções complexas.
Tamanho das instruções	Instruções de tamanho fixo.	Instruções de tamanho variável.
Complexidade	Complexidade no compilador.	Complexidade no código.
Modos de endereçamento	Poucos modos de endereçamento.	Muitos modos de endereçamento.
Quantidade de ciclos	Instruções em um único ciclo de máquina.	Instruções em múltiplos ciclos de máquina.
Paralelismo	Altamente paralelizado.	Fracamente paralelizado.
Acesso aos dados	Acesso aos dados via registradores.	Acesso aos dados via memória.
Microprogramação	Execução sem microprogramação.	Execução com microprogramação.
Quantidade de registradores	Vários registradores.	Poucos registradores.

RISC

- Unidades de controle mais simples, rápidas, baratas
- Instruções mais simples, pouca variedade, poucos endereços
- Consequentemente os circuitos internos são mais simples
- Menores, mais baratos, consomem pouca energia
  - A aplicação em aparelhos móveis influencia o desenvolvimento
- Como as instruções variam pouco, a unidade de controle consegue prever melhor quantos ciclos de máquina serão necessários
  - Ex.: ganho de desempenho no processamento paralelizado

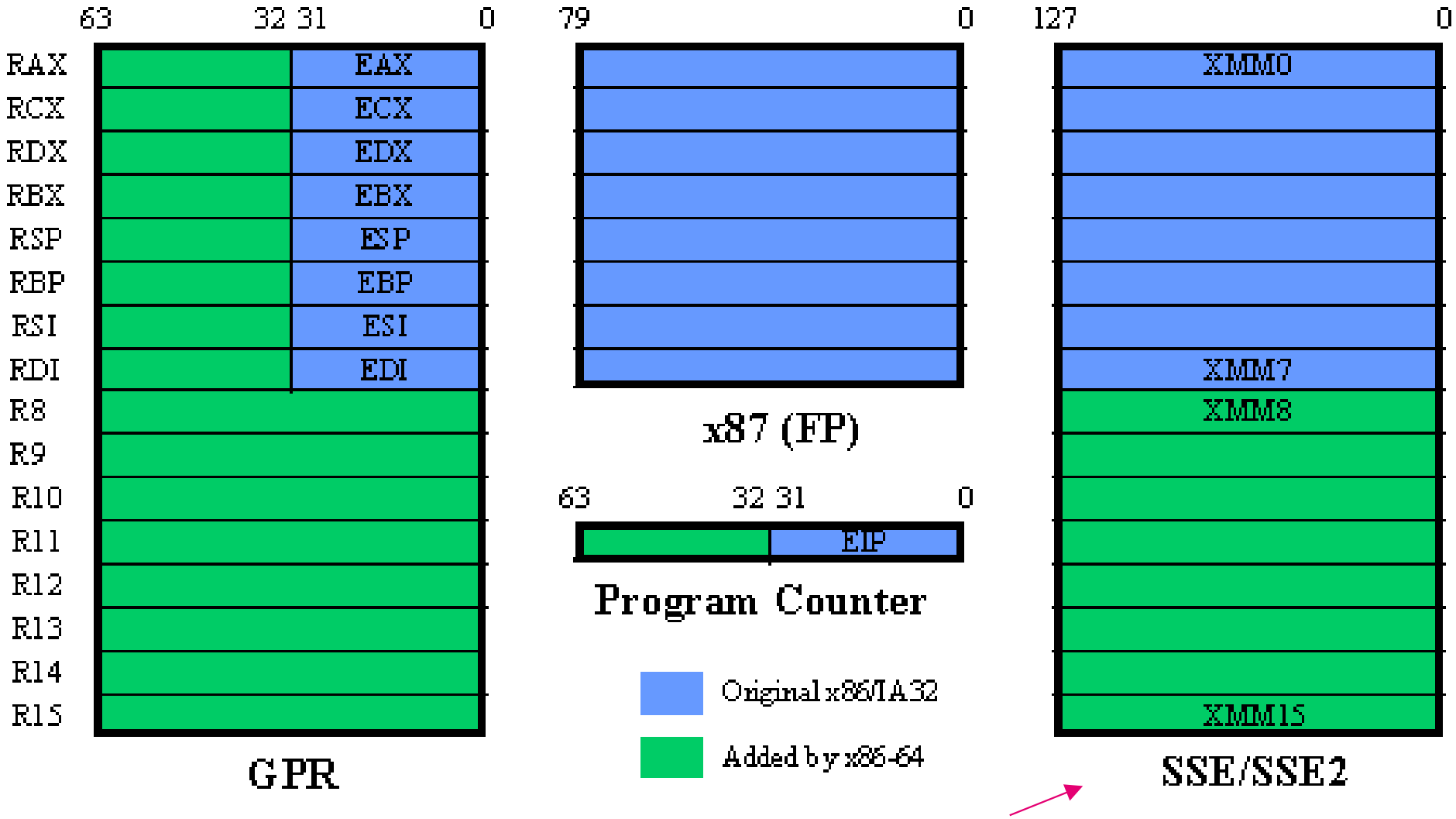
CISC

- Geralmente utilizado em máquinas de maior desempenho
  - Embora existam supercomputadores que utilizam RISC
- Grande quantidade de instruções
- Ênfase na construção de compiladores e linguagens de alto nível
- Maior densidade de código
- Geralmente são máquinas maiores, consumindo mais energia, e com maior capacidade de processamento
- Geralmente associado à arquitetura von Neumann
  - Internamente o processador pode utilizar arquitetura Harvard
    - Ex.: memória cache separada para dados e instruções

RISC x CISC

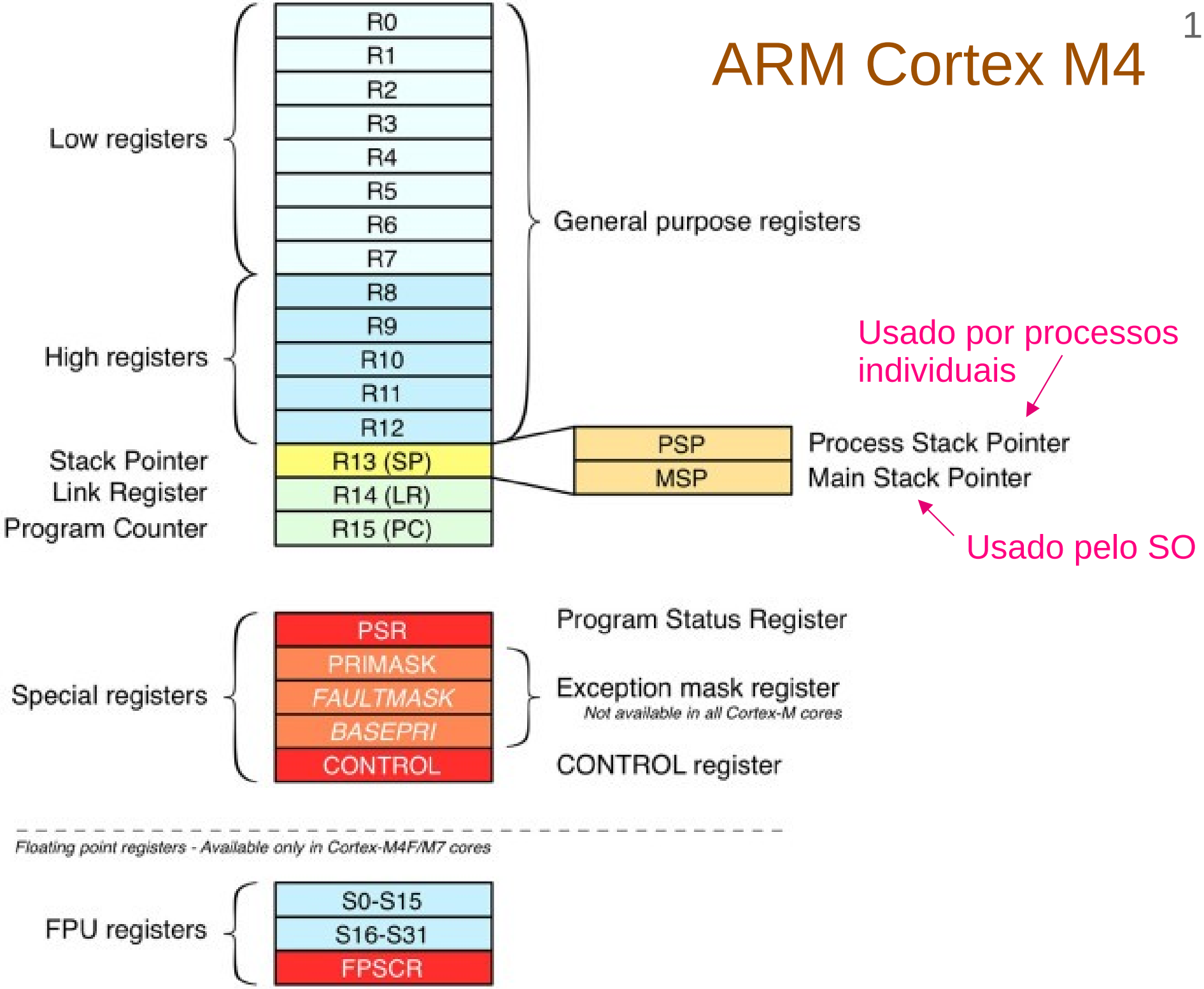
- Multiplicação a = a * b, com a no end1 e b no end2
  - CISC
    - MULT end1, end2
  - RISC
    - LOAD R1, end1
    - LOAD R2, end2
    - PROD R1, R2
    - STORE end1, R1

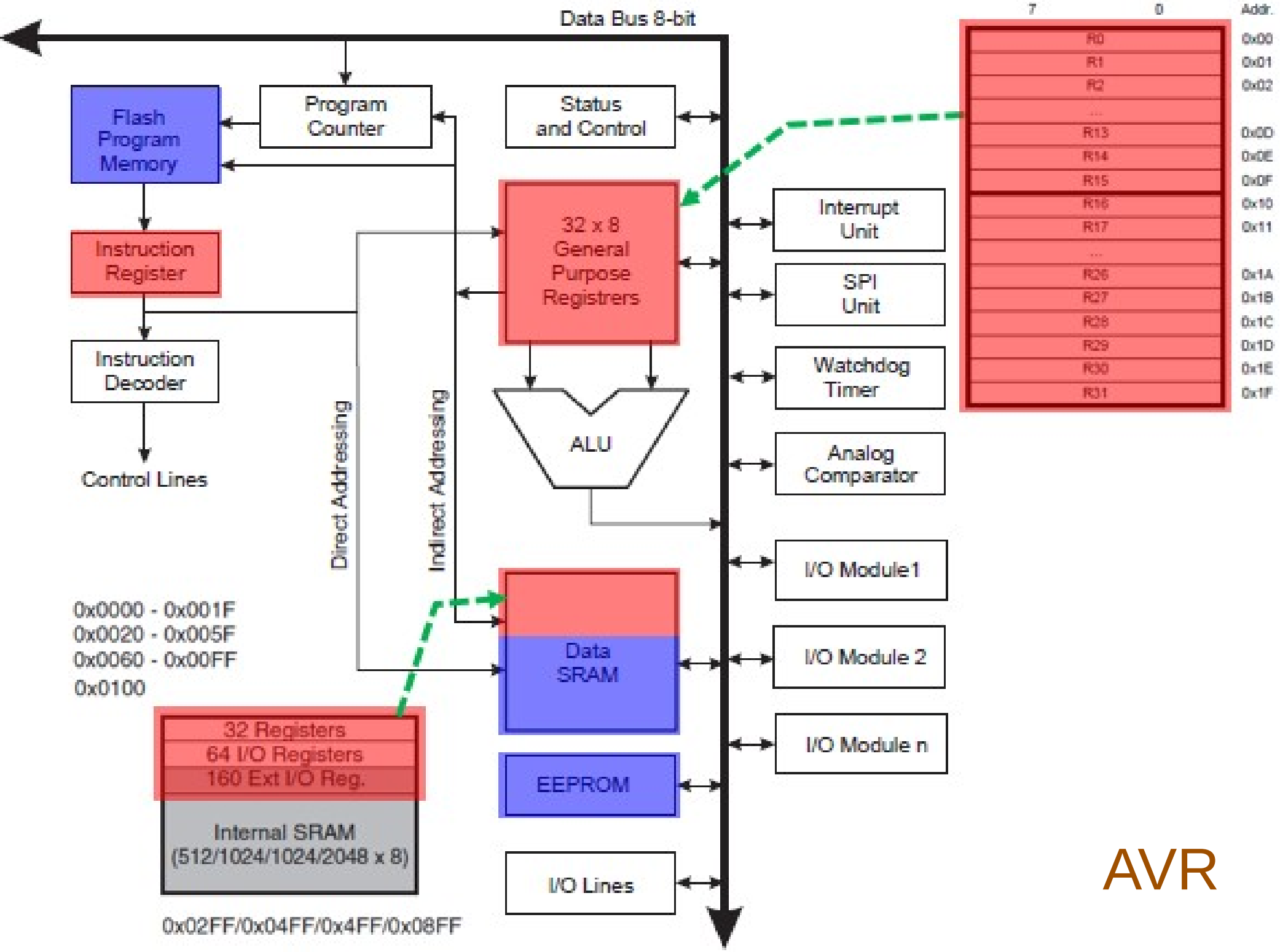
Arquitetura x86-64



- Mesma instrução usada em vários dados
- Move da memória para o registrador, executa a operação, move para a memória

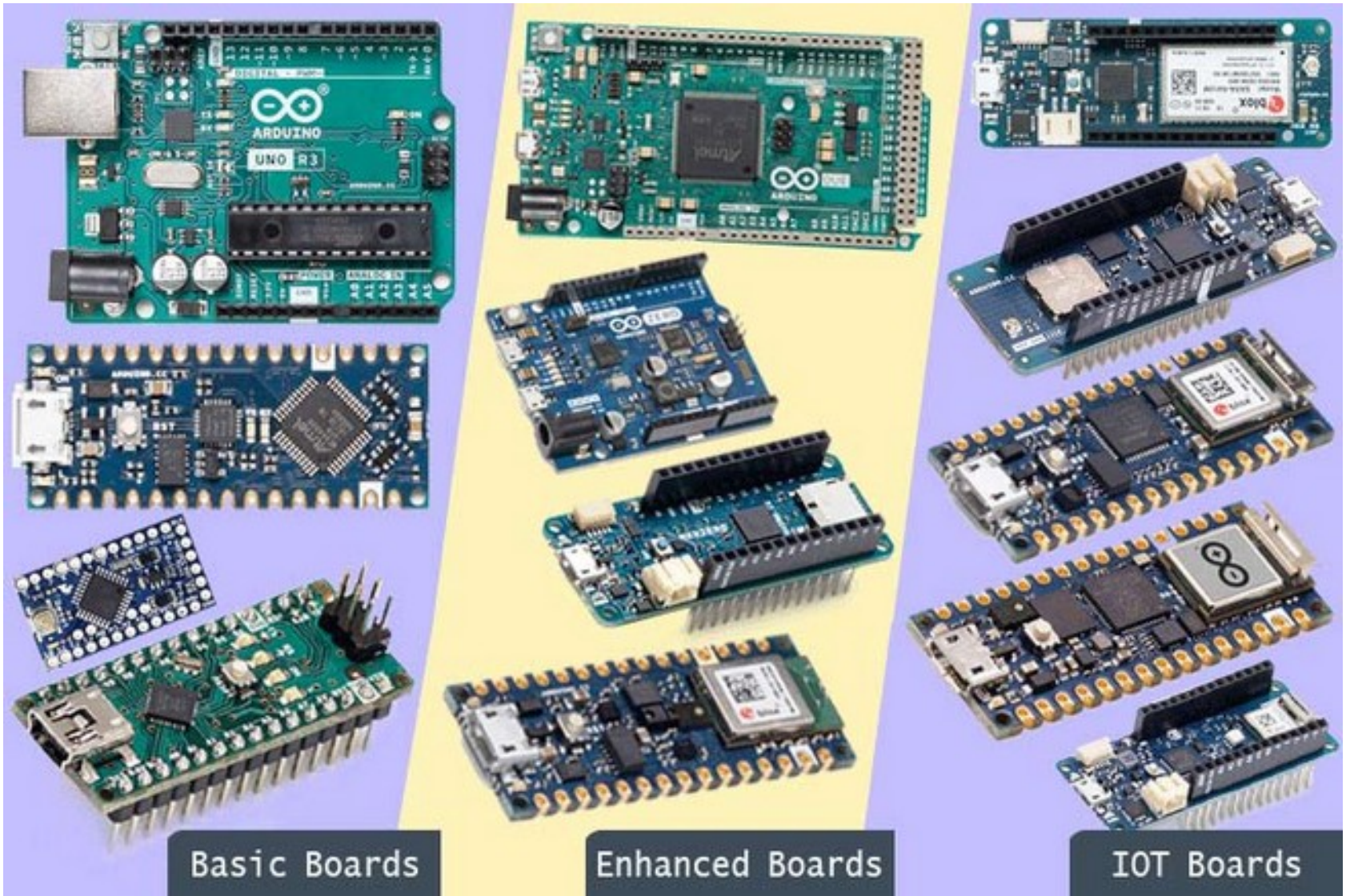
ARM Cortex M4

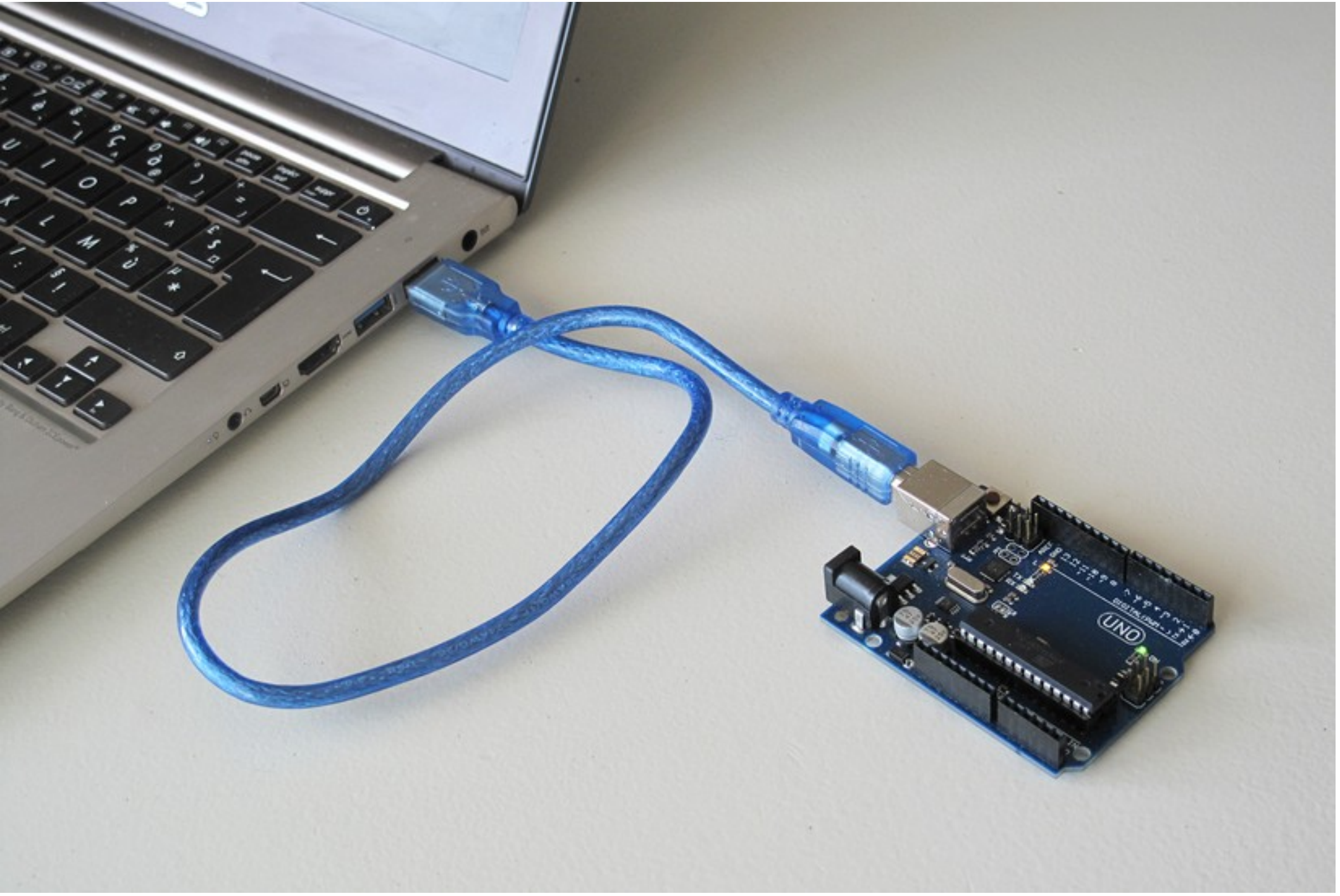


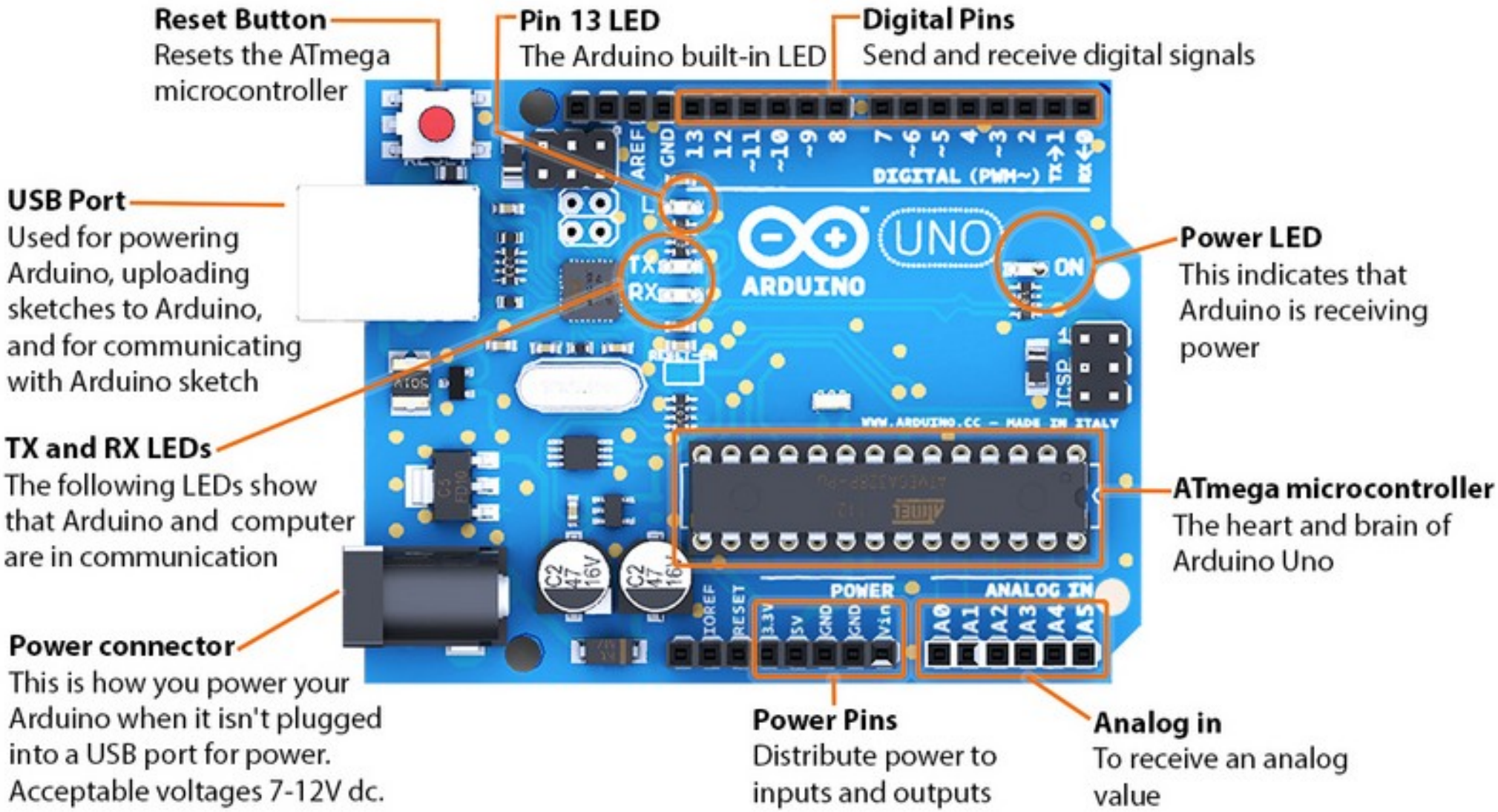

Data Bus 8-bit
Flash Program Memory
Program Counter
Status and Control
Instruction Register
Instruction Decoder
Control Lines
32 x 8 General Purpose Registrers
ALU
Direct Addressing
Indirect Addressing
Interrupt Unit
SPI Unit
Watchdog Timer
Analog Comparator
I/O Module1
I/O Module 2
I/O Module n
Data SRAM
EEPROM
I/O Lines
0x0000 - 0x001F
0x0020 - 0x005F
0x0060 - 0x00FF
0x0100
32 Registers
64 I/O Registers
160 Ext I/O Reg.
Internal SRAM
(512/1024/1024/2048 x 8)
0x02FF/0x04FF/0x4FF/0x08FF
7
0
Addr.
R0
R1
R2
...
R13
R14
R15
R16
R17
...
R26
R27
R28
R29
R30
R31
0x00
0x01
0x02
0x0D
0x0E
0x0F
0x10
0x11
0x1A
0x1B
0x1C
0x1D
0x1E
0x1F
AVR

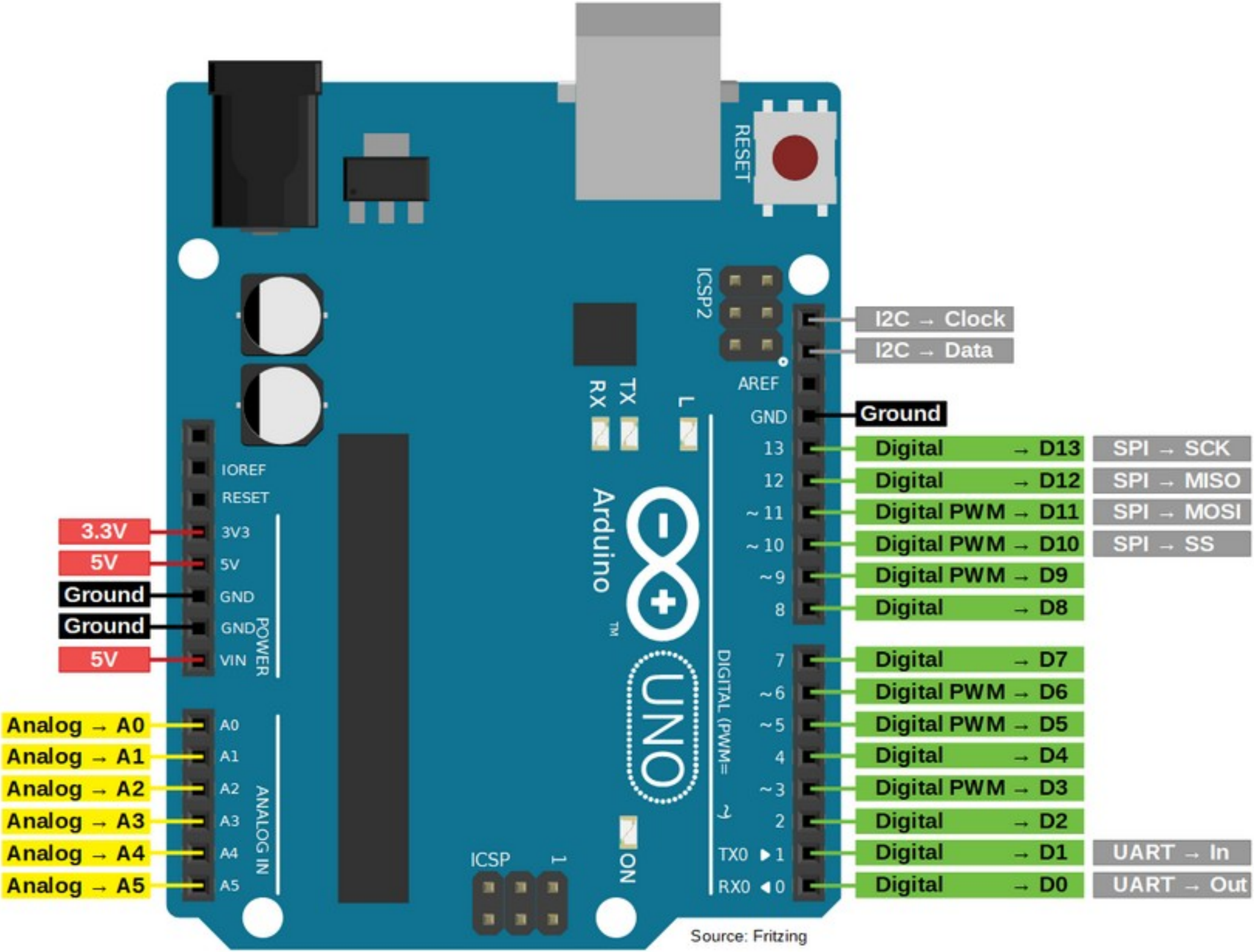
Arduino

- Arduino é uma empresa de hardware e software de código aberto
- Fabrica microcontroladores de placa única e kits



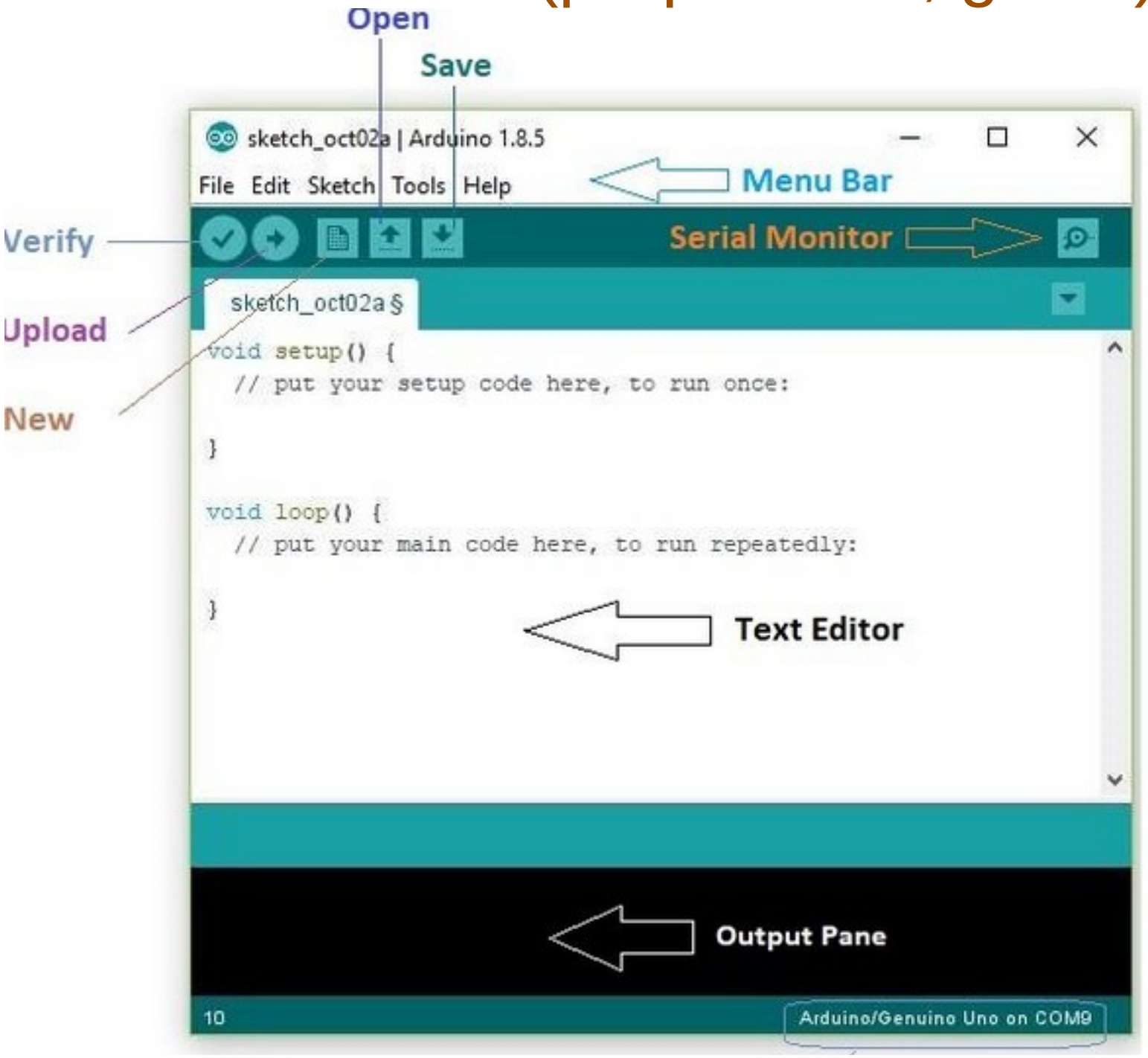

UNO


Reset Button
Resets the ATmega microcontroller
Pin 13 LED
The Arduino built-in LED
Digital Pins
Send and receive digital signals
USB Port
Used for powering Arduino, uploading sketches to Arduino, and for communicating with Arduino sketch
Power LED
This indicates that Arduino is receiving power
TX and RX LEDs
The following LEDs show that Arduino and computer are in communication
ATmega microcontroller
The heart and brain of Arduino Uno
Power connector
This is how you power your Arduino when it isn't plugged into a USB port for power. Acceptable voltages 7-12V dc.
Power Pins
Distribute power to inputs and outputs
Analog in
To receive an analog value
DIGITAL (PWM~)
UNO
ARDUINO
ON
ICSP
WWW.ARDUINO.CC – MADE IN ITALY
POWER
ANALOG IN
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
TX
RX
IOREF
RESET
3.3V
5V
GND
GND
Vin
A0
A1
A2
A3
A4
A5



Source: Fritzing

IDE (proprietária, grátis)



The Arduino Type and Port I'm using right now

Programação

- IDE Arduino
  - Usa um dialeto C/C++
    - Versão do GCC
- Bibliotecas
  - Grande quantidade de bibliotecas disponíveis
  - Torna fácil utilizar um periférico novo (produzido para o Arduino)
- Arduino Uno
  - Mais difundido
  - ATmega328p, 32 kB Flash, 2 kB RAM, 1 kB EEPROM
  - 14 pinos de I/O, 6 entrada analógicas, 16 MHz

IDE

- Um programa é chamado de “Sketch”.
- Pré-configurado com duas seções: Setup e Loop
  - Setup roda uma única vez
  - Loop fica rodando em um laço contínuo
- Um pino I/O pode ser configurado como entrada ou saída
  - pinMode(pino, modo)
    - pino varia de 0 a 13
- digitalRead(pino)
  - saída “1” (HIGH) para nível lógico alto

- digitalWrite(pino, valor)
  - HIGH: Nível alto, depende da tensão de alimentação
  - LOW: ~ 0 V

```
radounsigned char val;
void setup()
{
    pinMode(13, OUTPUT);
    pinMode(7, INPUT);
    pinMode(9, INPUT_PULLUP);
}
void loop ()
{
    val = digitalRead(7);
    digitalWrite(13, HIGH);
    delay (1000);
    digitalWrite(13, LOW);
    delay (1000);
}
```

CCP (Capture/Compare/PWM)

- Periférico do µC que permite ao programador controlar e medir o tempo de diversos eventos
- Funcionamento está associado diretamente aos **temporizadores** e **contadores**, internos do µC, que são usados como sinais de entrada para o sistema
- O ATmega328 possui 3 timers capazes de gerar até 6 sinais PWM nos pinos digitais nos pinos “~”
- PWM (Pulse Width Modulation): sinal digital com frequência fixa
  - O tempo em que o sinal fica em nível alto ou baixo pode variar

- O modo de captura permite a medição do tempo de duração de um determinado evento, monitorando o estado de um timer

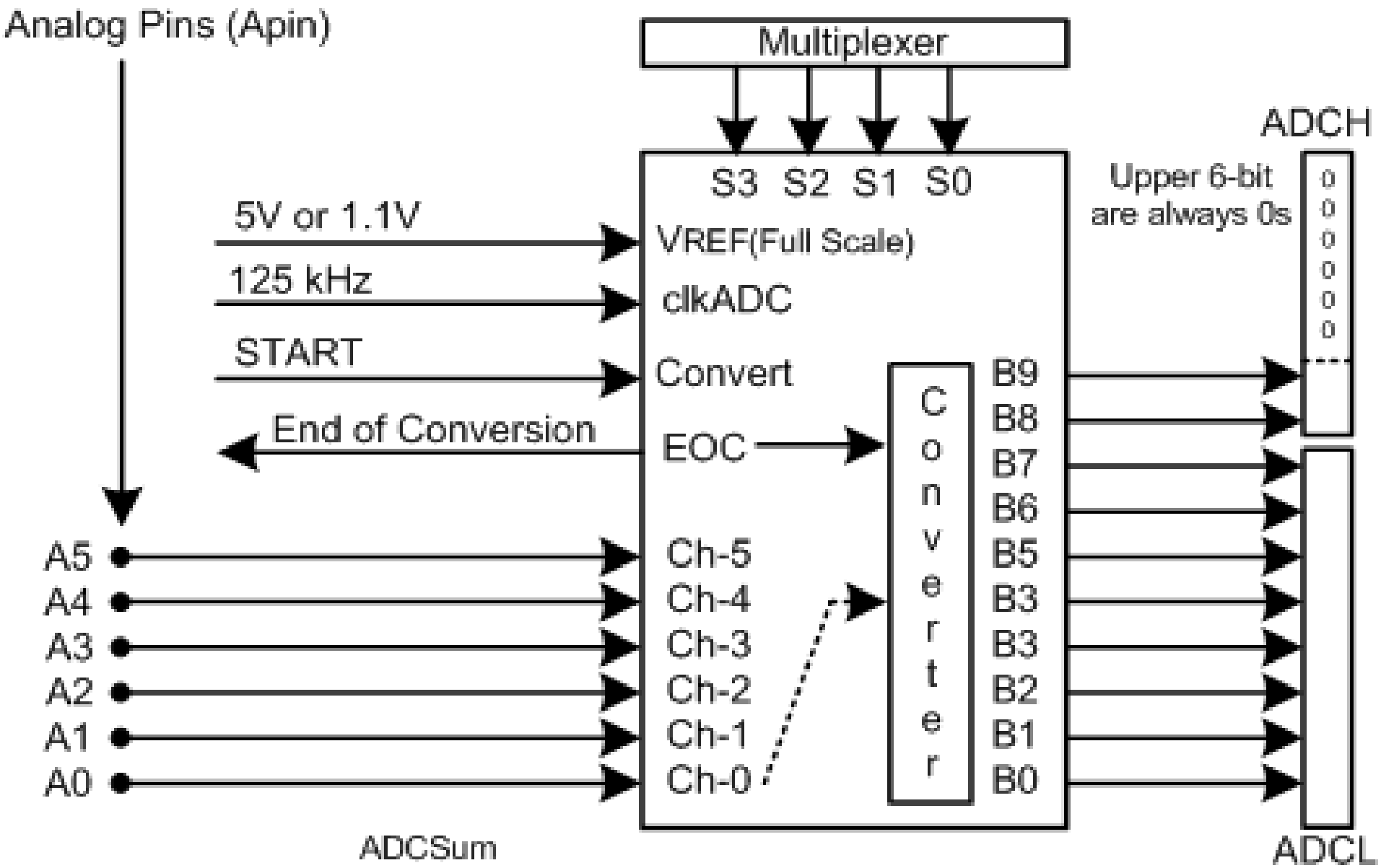
- O modo de comparação, compara o valor do timer com um valor previamente configurado, permitindo gerar ações quando o valor da contagem do temporizador igualar-se ao valor de referência

- O modo PWM gera sinais modulados por largura de pulso, permitindo ao programador escolher a frequência e o *ciclo de trabalho* a serem adotados, com base nos sinais gerados pelo temporizador

- Por serem partes da mesma estrutura, só é possível utilizar um sistema de cada vez

- analogWrite (pino, valor)
  - pino - onde será gerado o PWM
  - valor - razão cíclica do PWM, valor entre 0 e 255
- Alguns valores disponíveis no ambiente de desenvolvimento
  - 980 Hz para os pinos 5 e 6
  - 490 Hz para 3, 9, 10, 11
- Conversor analógico-digital (A/D)
  - ATmega328 possui um único, de 10 bits
  - Pode ser multiplexado em 6 canais (**`analogRead (pino)`**)
    - Um único canal por vez, 0 a 5
  - Retorna um número de 0 a 1023
  - O intervalo de tensões depende da tensão de alimentação

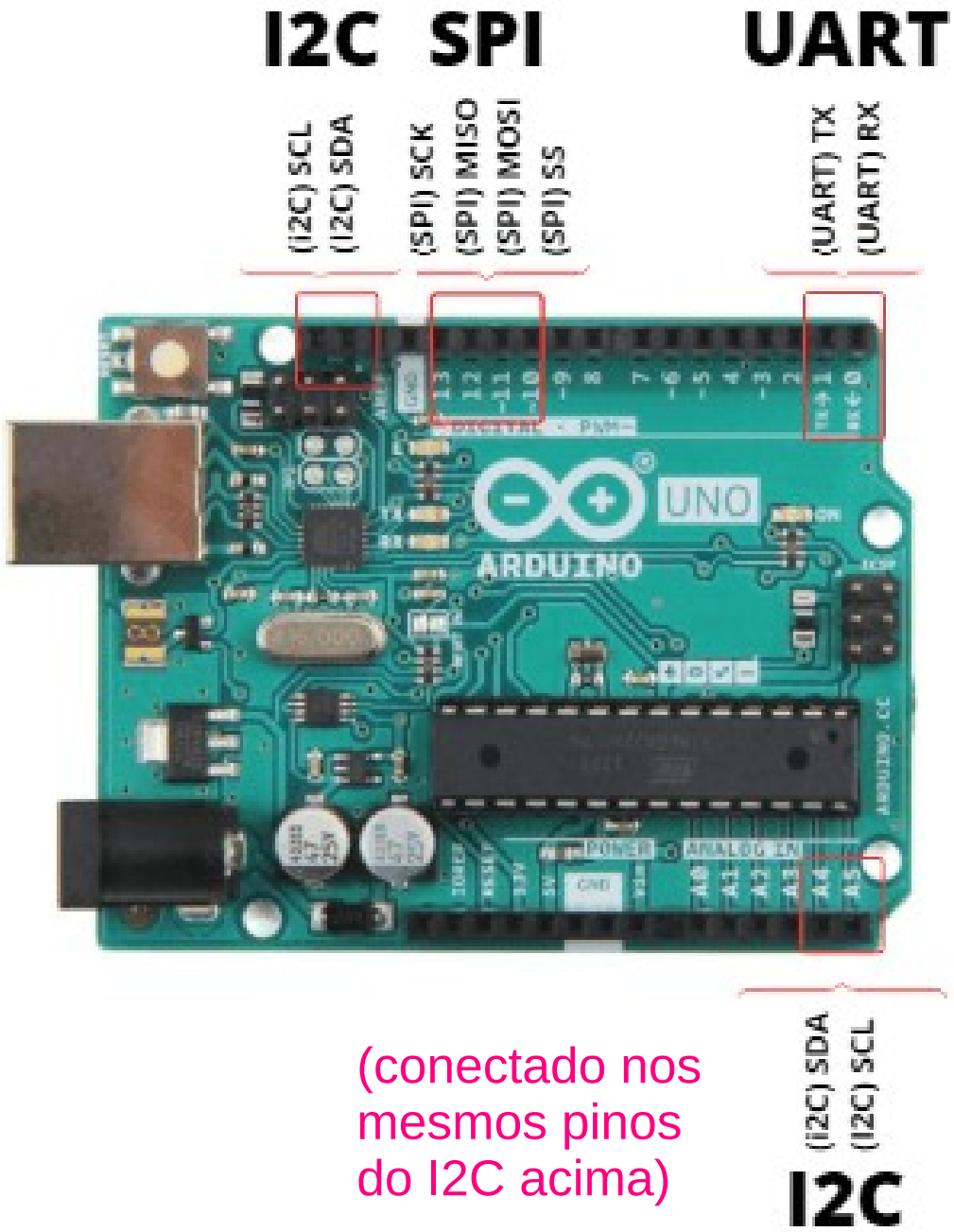
Módulo ADC do ATmega328P



- As funções analógicas do Arduino não necessitam de inicialização

```
int PinoSensor = A2;
int PinoPWM = 5;
int ValorSensor;
void setup()
{
    // sem inicialização
}
void loop()
{
    ValorSensor = analogRead(PinoSensor);
    analogWrite(PinoPWM, ValorSensor/4);
    delay(10);
}
```

Comunicação

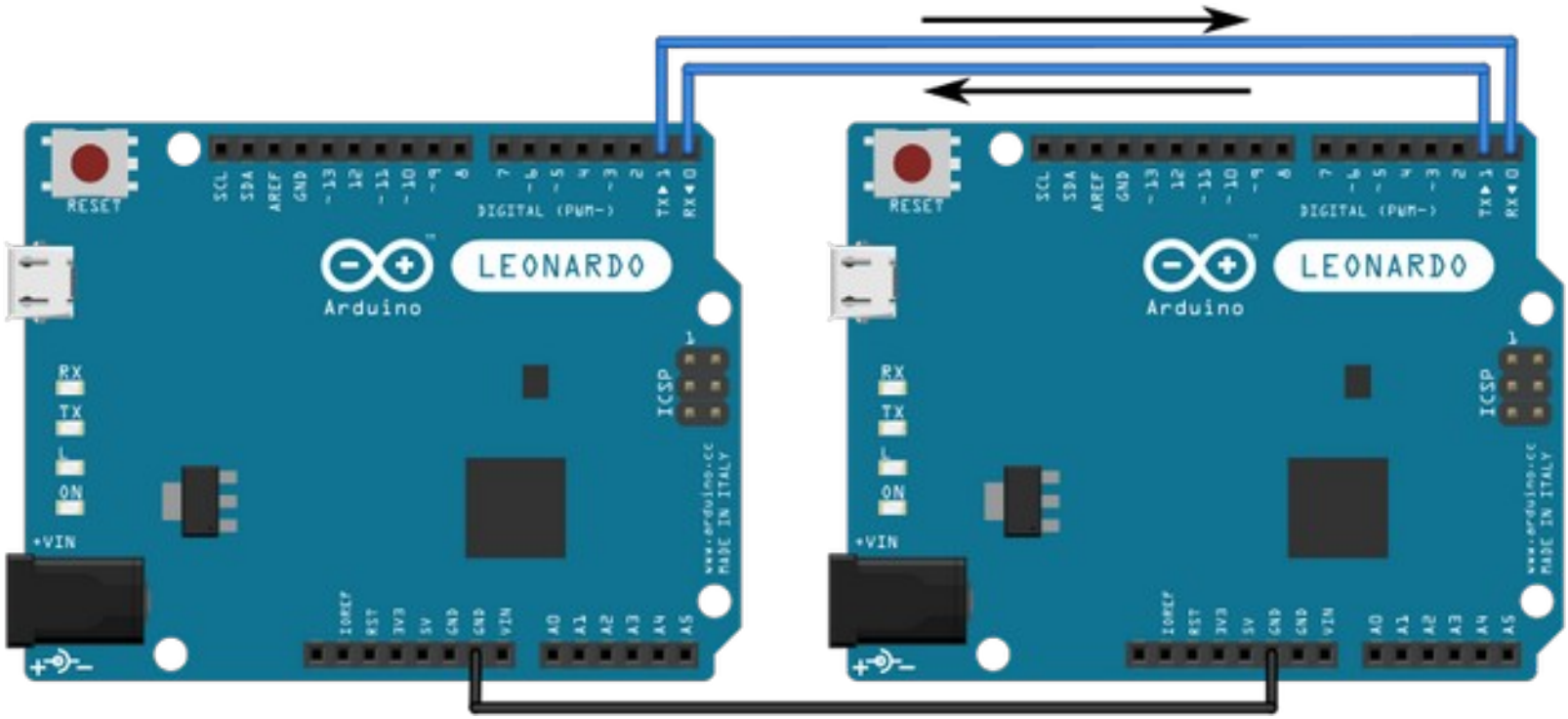


USB

(conectado nos mesmos pinos do I2C acima)

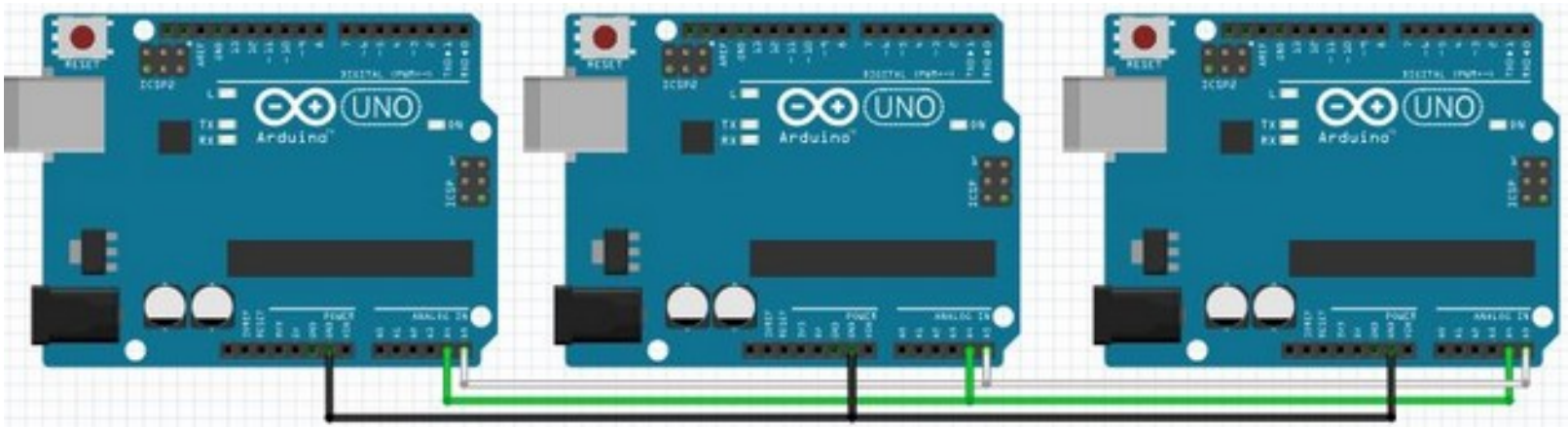
UART - Universal Asynchronous Serial Receiver Transmitter

- 2 fios, 1 Mbps
- Apenas 2 dispositivos
  - Síncrono - clock, transmissão (pino 1), recepção (pino 0)
  - Assíncrono - terra, transmissão, recepção



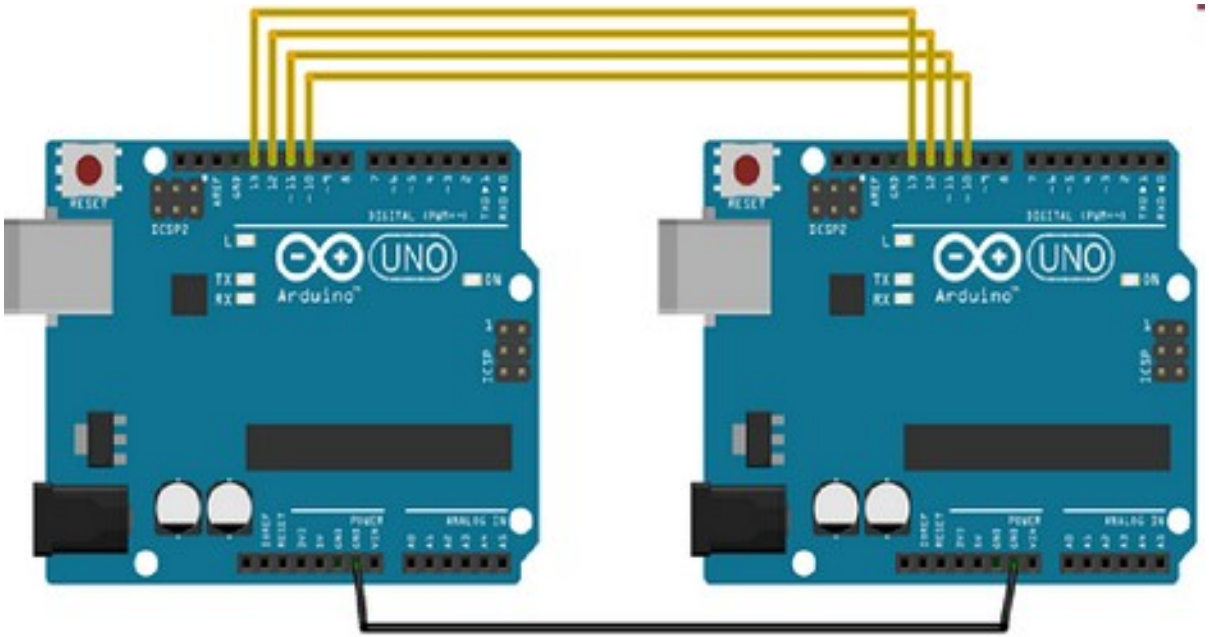
I2C - Inter-Integrated Circuit

- 2 fios, 0,1 Mbps
- multi-master, multi-slave, até 128 devices



SPI - Serial Peripheral Interface

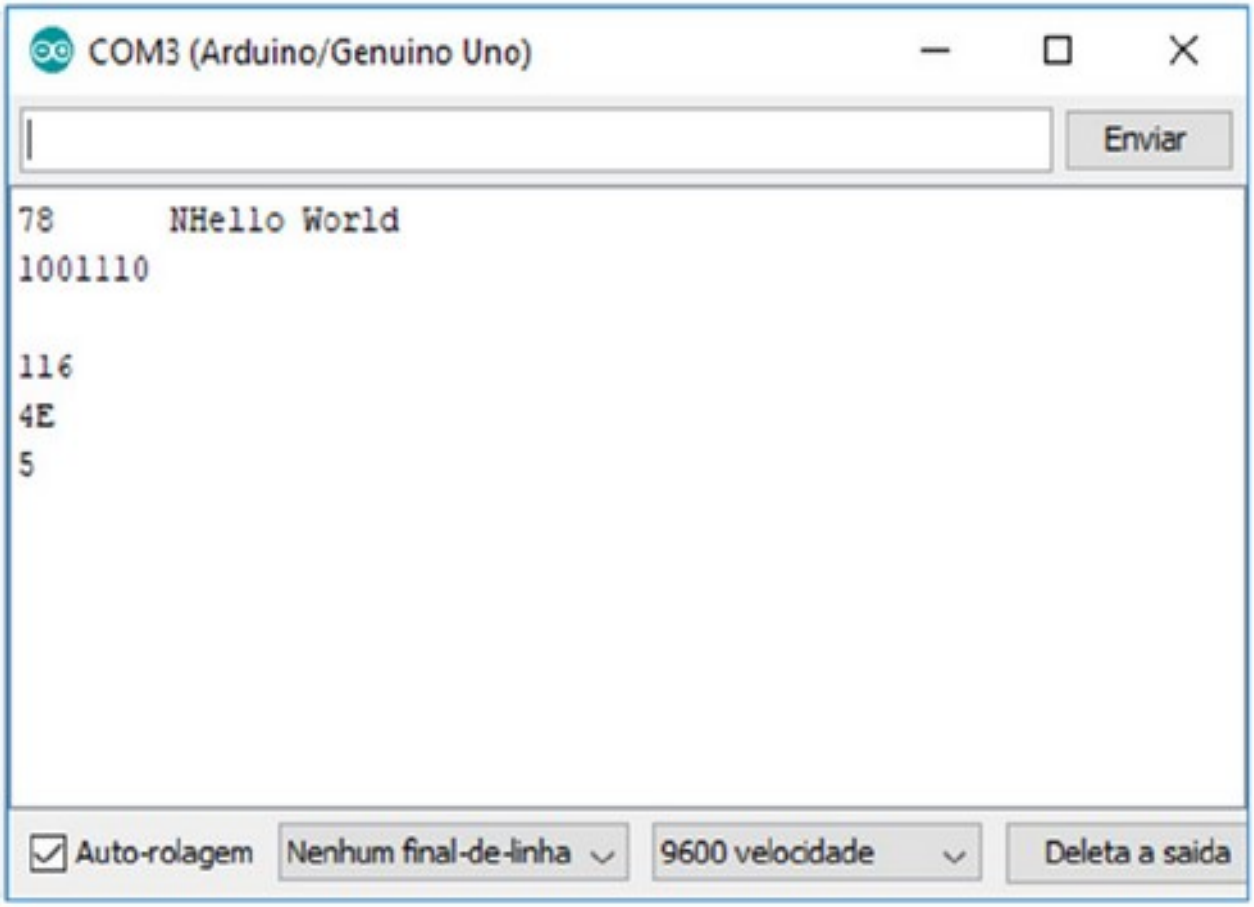
- 4 fios, 4 Mbps
- 1 master, ~ 17 slaves no uno limitado a distâncias e capacitâncias



Exemplo

```
int a = 5;
void setup()
{
  Serial.begin(9600);

  Serial.print(78);
  Serial.print("\t");
  Serial.print('N');
  Serial.println("Hello World");
  Serial.println(78,BIN);
  Serial.println("");
  Serial.println(78,OCT);
  Serial.println(78,HEX);
  Serial.println(a);
}
```



Arduino mandando comunicação para o PC

PC recebendo a comunicação em um terminal